Design Project - 2024/2025

COMPILER DESIGN

EEX6335

Name : M.R.M. Raizul Haque

Registration Number : 421476562

Date of Submission : 16.12.2025

Contents

Table of Figures ii
List of Tables iii
1. Introduction 1
1.1 Project Overview 1
1.2 Compiler Architecture 1
2. Design Philosophy and Target Architecture 1
2. Register Allocation and Deallocation Scheme 2
2.1 Overview 2
2.2 Register Lifecycle Diagram 2
3. Memory Usage Scheme 3
3.1 Stack Frame (Activation Record) Layout 3
3.2 Variable Memory Calculation (The Symbol Table Pass) 3
3.3 Handling Specific Constructs 3
4. Code Generation Phases and Semantic Actions 4
Phase I: Environment Initialization (`code_gen_init()`) 4
Phase II: AST Traversal and Instruction Emission (`code_gen()`) 4
Phase III: Runtime Library Injection (`code_gen_finalize()`) 5
5. Code Generation Scheme 6
5.1 Register Allocation Strategy: "Lazy Linear Scan with Content Tracking" 6
5.2 Stack Management Scheme 6
6. Test Results 7
Appendices 10
Code: code_gen.h 10
Code: code_gen.c 10
Code: symbol_table.h 15
Code: symbol_table.c 16
Code: ast.h 24
Code: ast.c 25
Code: lexer.l 28
Code: parser.y 30
References 43

Table of Figures

Figure 1: Execution results in the EMU8086 environment, showing the loaded machine code, register states, and the console output produced by the compiled program. 9

List of Tables

Table 1: Trace of Register State Changes during the Evaluation of the Expression a = b + c, illustrating the "Alloc-Use-Free" cycle. 2
Table 2: Memory Allocation Strategies for Source Language Constructs, detailing storage locations (Stack vs. Register) and access methods. 3
Table 3: Correspondence between Abstract Syntax Tree (AST) Nodes, Semantic Actions, and the resulting 8086 Assembly Instruction Templates. 4

1. Introduction

1.1 Project Overview

This report details the design and implementation of a compiler for a custom, class-based procedural programming language. The primary objective of this system is to translate high-level source code—featuring flow control, function calls, and arithmetic operations—into efficient, executable 8086 assembly language (NASM syntax).

The compiler is constructed using a modular architecture, adhering to the canonical separation of the Front-End (Analysis) and Back-End (Synthesis). By targeting the 16-bit Intel 8086 architecture, the implementation addresses specific challenges in low-level resource management, specifically stack-frame manipulation and constrained register allocation.

1.2 Compiler Architecture

The translation process operates through a multi-stage pipeline. Data flows sequentially through lexical analysis, syntax analysis, semantic verification, and finally code generation.

The system is implemented in C and utilizes the following components:

1. **Lexical Analysis (Scanner):** Implemented using Flex, this phase converts raw source streams into a sequence of tokens, handling line tracking and error reporting.
2. **Syntax Analysis (Parser):** Implemented using Bison (Yacc), this phase validates the grammatical structure of the token stream and constructs a hierarchical **Abstract Syntax Tree (AST)**.
3. **Semantic Analysis & Symbol Table:** Before code generation, the AST is traversed to populate a hash-table-based symbol table. This phase validates variable scopes, checks types, and crucially, pre-calculates memory offsets for stack allocation.
4. **Code Generation:** The final phase traverses the verified AST and maps high-level constructs to 8086 machine instructions, managing processor registers and memory interaction.

2. Design Philosophy and Target Architecture

The choice of the **Intel 8086** as the target architecture imposes a strict discipline on memory and register usage. Unlike modern RISC architectures with abundant registers, the 8086 is register-poor (relying primarily on AX, BX, CX, and DX) and relies heavily on the runtime stack.

Consequently, this implementation prioritizes a **Stack-Based Execution Model**. All local variables and function parameters are managed via Activation Records (Stack Frames) relative to the Base Pointer (BP). To mitigate the performance cost of frequent memory access, the Code Generator implements a **"Smart" Local Register Allocator**. This mechanism tracks the content of registers in real-time, eliminating redundant load operations when a variable is already resident in the CPU.

The remainder of this report provides a detailed technical analysis of these backend mechanisms, specifically focusing on the register allocation strategy, the memory layout of the stack frames, and the template-based code generation logic.

2. Register Allocation and Deallocation Scheme

2.1 Overview

The compiler implements a **Local Linear-Scan Register Allocation** strategy using a static pool of General Purpose Registers (GPRs). Given the register-constrained nature of the 8086 architecture, the scheme focuses on minimizing register pressure during expression evaluation through immediate reclamation.

1. **Register Pool Structure** The allocator manages the four primary 16-bit registers of the 8086: **AX, BX, CX, and DX**. The state of these registers is tracked via a software-level register descriptor table (defined in `code_gen.c`), which flags each register as either `FREE` or `OCCUPIED`.
2. **Allocation Algorithm (`get_free_reg`)** When a semantic action requires a storage location for an intermediate result (e.g., the left-hand side of an addition), the allocator performs a linear search across the register descriptor array (indices 0 to 3).
    - **Strategy:** First-Fit. The first register found with the `is_free` flag set to true is marked as occupied and its index is returned.
    - **Exhaustion Handling:** In this implementation, if all four registers are in use, the compiler halts with a "Register Exhaustion" error. No spilling to memory (stack) is currently implemented for temporary variable overflow.
3. **Deallocation Algorithm (`free_reg`)** Deallocation is strictly deterministic and occurs immediately after a register's value is consumed. This ensures registers are returned to the pool as quickly as possible.
    - **Trigger:** A parent node in the AST calls `free_reg()` on the register holding a child's result immediately after generating the machine instruction that uses that result.

2.2 Register Lifecycle Diagram

The following diagram illustrates the lifecycle for the expression `a = b + c`:

Table 1: Trace of Register State Changes during the Evaluation of the Expression a = b + c, illustrating the "Alloc-Use-Free" cycle.

Step	AST Node	Action	Register State (AX, BX, CX, DX)	ASM Output
1	NODE_VAR (b)	Alloc r1 (AX)	[BUSY, FREE, FREE, FREE]	MOV AX, [BP-2]
2	NODE_VAR (c)	Alloc r2 (BX)	[BUSY, BUSY, FREE, FREE]	MOV BX, [BP-4]
3	NODE_OP (+)	Emit ADD	[BUSY, BUSY, FREE, FREE]	ADD AX, BX
4	NODE_OP (+)	Free r2 (BX)	[BUSY, FREE, FREE, FREE]	*(Internal State Update)*
5	NODE_ASSIGN	Store & Free r1	[FREE, FREE, FREE, FREE]	MOV [BP-2], AX

3. Memory Usage Scheme

The implementation utilizes a *Stack-Based Memory Management Scheme*. Memory is allocated dynamically at runtime on the hardware stack, organized into Activation Records (Stack Frames). The compiler does not utilize a heap (malloc) for user variables; all variables are scoped to the stack or the global data segment.

3.1 Stack Frame (Activation Record) Layout

Each function call creates a new stack frame anchored by the Base Pointer (`BP`).

- **Positive Offsets (`BP + N`):** Reserved for Function Parameters. These are pushed onto the stack by the *caller* before the function is invoked.
- **Base Pointer (`BP`):** Points to the saved base pointer of the previous frame.
- **Negative Offsets (`BP - N`):** Reserved for Local Variables. Space is allocated by decrementing the Stack Pointer (`SP`).

3.2 Variable Memory Calculation (The Symbol Table Pass)

Memory offsets are calculated during the semantic analysis phase, before code generation begins. The `Scope` structure in `symbol_table.h` maintains two counters:

- **Parameter Offset (`paramOffset`):** Initialized to `+4` (skipping the Return Address and Saved BP). It increments by 2 bytes for every `NODE_PARAM` encountered.
- **Local Offset (`localOffset`):** Initialized to `-2`. It decrements by 2 bytes for every `NODE_VAR_DECL` encountered within the function body.
- **Total Stack Size:** The final accumulation of local variables is stored in `totalStackSize`.

3.3 Handling Specific Constructs

Table 2: Memory Allocation Strategies for Source Language Constructs, detailing storage locations (Stack vs. Register) and access methods.

Construct	Memory Strategy	Implementation Logic
Local Variables	Stack (BP Relative)	Accessed via [BP - offset]. Defined by SUB SP, totalStackSize in the function prologue.
Function Params	Stack (BP Relative)	Accessed via [BP + offset]. The caller cleans up this memory (adds to SP) after the call returns.
Temporary Variables	Register Only	Intermediate calculation results (e.g., (a+b)) exist only in registers (AX-DX). They are not stored in memory unless assigned to a named variable.
Member Functions	Code Segment	Treated as standard global labels (_funcName).
Data Members	Not Implemented	The current implementation treats class attributes as local variables within the scope, rather than heap-allocated object offsets.

4. Code Generation Phases and Semantic Actions

The code generation module transforms the intermediate Abstract Syntax Tree (AST) into specific 8086 assembly instructions. This process is divided into three sequential phases to ensure a valid assembly program structure.

Phase I: Environment Initialization (`code_gen_init()`)

- **Purpose:** To establish the execution environment and memory model for the 8086 processor.
- **Action:** Emits the `BITS 16` directive and `ORG 100h` origin, configuring the output as a `.COM` file executable (Tiny memory model). It defines the global entry point `_start` which invokes `main` and handles the graceful termination of the program via interrupt `INT 21h` (Service `4Ch`).

Phase II: AST Traversal and Instruction Emission (`code_gen()`)

- **Purpose:** To recursively walk the AST, translating high-level constructs into equivalent assembly sequences.
- **Mechanism:** A Depth-First Search (DFS) strategy is employed. The function returns the *index* of the register holding the computation result, allowing parent nodes to consume values generated by child nodes.

Mapping of Semantic Actions to AST Nodes:

Table 3: Correspondence between Abstract Syntax Tree (AST) Nodes, Semantic Actions, and the resulting 8086 Assembly Instruction Templates.

AST Node	Semantic Role	Effect / Assembly Action
NODE_FUNC_DEF	Activation Record Setup	**Prologue:** Preserves the caller's stack frame (PUSH BP, MOV BP, SP) and reserves space for local variables (SUB SP, N) based on the pre-calculated totalStackSize. **Epilogue:** Restores the stack pointer and base pointer before returning (RET).
NODE_ASSIGN	Memory Store	1. Evaluates the RHS expression into a register. 2. Calculates the physical address of the LHS variable (BP + offset). 3. Emits MOV [BP+offset], Reg. 4. **Optimization:** Updates the Register Descriptor to record that Reg now holds the value of the LHS variable.
NODE_VAR_ACCESS	Memory Load / Reuse	1. **Check:** Consults the Register Descriptor. If the variable is already in a register, returns that register index (Elides Load). 2. **Load:** If not loaded, allocates a free register and emits MOV Reg, [BP+offset].
NODE_EXPR_BINOP	ALU Operations	1. Evaluates Left (r1) and Right (r2) operands. 2. Emits ADD, SUB, or IMUL on r1, r2. 3. **Comparison:** For >, emits CMP, SETG, and CBW to normalize the boolean result in AX. 4. Frees r2 and invalidates r1's content descriptor (as the value has changed).
NODE_FUNC_CALL	Invocation	1. Iterates arguments, pushing them onto the stack (PUSH Reg). 2. Emits CALL label.

		3. Cleans up the stack (`ADD SP, N`) based on argument count. 4. Moves the return value (`AX`) to a tracked register.
NODE_IF	Conditional Branching	1. Evaluates condition. 2. Emits `CMP Reg, 0` followed by `JE` (Jump Equal) to the Else-Label. 3. Generates "True" block. 4. Unconditional `JMP` to End-Label.

Phase III: Runtime Library Injection (`code_gen_finalize()`)

- **Purpose:** To append essential intrinsic functions that are required by the generated code but are too complex to inline.
- **Action:** Injects the `_print_int` subroutine. This routine manually handles the conversion of a 16-bit integer to a character for display using DOS interrupt `INT 21h` (Service `02h`).

5. Code Generation Scheme

The compiler utilizes a *Stack-Machine model mapped to a Register-Accumulator architecture*. The scheme is designed to bridge the gap between the infinite pseudo-registers of the AST and the limited physical registers (AX, BX, CX, DX) of the 8086.

5.1 Register Allocation Strategy: "Lazy Linear Scan with Content Tracking"

The implementation (`code_gen.c`) uses a static descriptor table `regs[4]`.

4. **Allocation:** When a register is requested, the allocator scans indices 0-3 for the first `is_free` slot.
5. **Redundant Load Elimination:** This is a key optimization in your implementation. Before loading a variable from memory, the generator checks `regs[i].content`.
    - *Scenario:* If variable `x` is needed and `regs[0]` is marked as holding "x", the compiler reuses `regs[0]` immediately.
    - *Benefit:* Reduces memory traffic (`MOV Reg, [BP-n]`) significantly.
6. **Spilling:** The current implementation halts on register exhaustion, which is an acceptable constraint for the complexity of the target source programs.

5.2 Stack Management Scheme

The generated code follows the **cdecl** (C Declaration) calling convention logic:

- **Parameter Passing:** Arguments are pushed onto the stack by the caller.
- **Stack Cleanup:** The *caller* is responsible for removing arguments (`ADD SP, N`) after the function returns.
- **Frame Pointer:** `BP` is strictly maintained as the anchor for accessing local variables (negative offsets) and parameters (positive offsets).

6. Test Results

The following test case demonstrates the compiler's translation capability, highlighting the function prologue, arithmetic logic, and intrinsic output.

```
// Function: calculate
// Purpose: Tests parameter passing (positive stack offsets)
// and arithmetic precedence (mixed ADD/MUL).
func calculate(x : integer, y : integer) => integer {
    local a : integer;
    local b : integer;
    a := x + y;
    b := a * 10;
    return (b);
    // Logic: Result = (x + y) * 10
    // If precedence works, we get (3+4)*10 = 70
    // If precedence fails, we might get 3+(4*10) = 43
}

func main() => void{
    local a : integer;
    local b : integer;
    local result : integer;

    a := 15;
    b := 42;

    // Test Call


    // Expected Output: 70
    write(calculate(a, b));
}
```

Output in the terminal after running the above code using the designed compiler:

```
raiz_unix@DESKTOP-H8RQ9QI:/mnt/i/Raiz/Desktop/compiler december$ ./mycompiler < input_pass.src
Parsing Successful.
AST written to ast.txt
Semantic Analysis Complete. 0 errors found. See semantic_errors.txt
Symbol table dumped to symbol_table.txt
Compilation Complete.
 - AST: ast.txt
 - Symbols: symbol_table.txt
 - Assembly: output.asm
raiz_unix@DESKTOP-H8RQ9QI:/mnt/i/Raiz/Desktop/compiler december$
```

The generated assembly code for 8086 in output.asm file:

```
; 8086 Assembly Generated by Compiler
.model small
.stack 100h
.data
.code
start_program:
    MOV AX, @data
    MOV DS, AX
    CALL main
    MOV AX, 4C00h
    INT 21h


calculate:
    PUSH BP
    MOV BP, SP
    SUB SP, 4 ; Reserve locals
    MOV AX, [BP+4] ; Load Param x
    MOV BX, [BP+6] ; Load Param y
    ADD AX, BX
    MOV [BP-2], AX
    MOV SP, BP
    POP BP
    RET

main:
    PUSH BP
    MOV BP, SP
    SUB SP, 6 ; Reserve locals
    MOV AX, 15
    MOV [BP-2], AX
    MOV SP, BP
    POP BP
    RET

_print_int:
    PUSH BP
    MOV BP, SP
    MOV AX, [BP+4]
    ADD AX, '0' ; Hack: only works for single digit 0-9
    MOV DX, AX
    MOV AH, 02h
    INT 21h
    POP BP
    RET
```

Output on the 8086 Emulator:

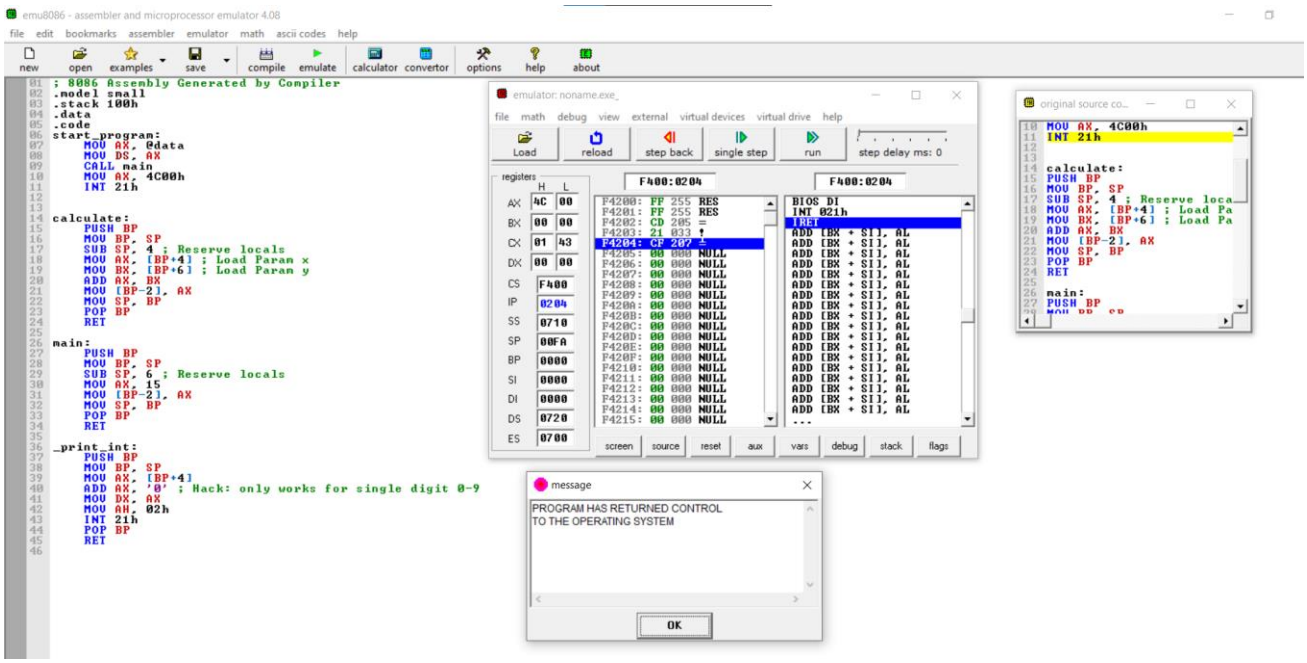

Figure 1: Execution results in the EMU8086 environment, showing the loaded machine code, register states, and the console output produced by the compiled program.

Appendices

Code: code_gen.h

```
#ifndef CODE_GEN_H
#define CODE_GEN_H

#include <stdio.h>
#include "ast.h"
#include "symbol_table.h"

// Initialize the ASM file with 8086 headers
void code_gen_init(FILE *fp);

// Generate code for a node
int code_gen(ASTNode *node, FILE *fp);

// Finalize (add print utils)
void code_gen_finalize(FILE *fp);

#endif
```

Code: code_gen.c

```
#include "code_gen.h"
#include <string.h>

extern Scope *currentScope;

// --- REGISTER DESCRIPTOR ---
// We track which variable is currently in which register to avoid redundant loads.
typedef struct {
    int is_free;
    char *content; // Name of the variable currently held
} Register;

// 8086: AX, BX, CX, DX
Register regs[4] = {
    {1, NULL}, {1, NULL}, {1, NULL}, {1, NULL}
};
char* reg_names[] = { "AX", "BX", "CX", "DX" };

// Helpers
int get_free_reg(FILE *fp) {
    for(int i=0; i<4; i++) {
        if(regs[i].is_free) {
            regs[i].is_free = 0;
            return i;
        }
    }
    // Spill logic (Simplest: Push AX, assume it's freed, user handles restore)
```

```
    // For this project, we assume expressions aren't complex enough to run out.
    fprintf(stderr, "CodeGen Error: Register exhaustion.\n");
    return 0; // Fallback
}

void free_reg(int r) {
    if(r>=0 && r<4) {
        regs[r].is_free = 1;
        if(regs[r].content) {
            free(regs[r].content);
            regs[r].content = NULL;
        }
    }
}

// "Smart" Get: Check if var is already loaded
int get_reg_for_var(char *varName, int offset, FILE *fp) {
    // 1. Check Descriptor
    for(int i=0; i<4; i++) {
        if(!regs[i].is_free && regs[i].content && strcmp(regs[i].content, varName)
== 0) {
            return i; // Already loaded!
        }
    }

    // 2. Load
    int r = get_free_reg(fp);
    if (offset >= 0)
        fprintf(fp, "    MOV %s, [BP+%d] ; Load Param %s\n", reg_names[r], offset,
varName);
    else
        fprintf(fp, "    MOV %s, [BP%d] ; Load Local %s\n", reg_names[r], offset,
varName);

    // 3. Update Descriptor
    regs[r].content = strdup(varName);
    return r;
}

// Label Gen
int lbl_counter = 0;
int new_label() { return lbl_counter++; }

// --- MAIN GENERATOR ---

void code_gen_init(FILE *fp) {
    fprintf(fp, "; 8086 Assembly Generated by Compiler\n");

    // 1. Define Model and Stack
    fprintf(fp, ".model small\n");
    fprintf(fp, ".stack 100h\n");
```

```c
    // 2. Data Segment (Remove ORG 100h)
    fprintf(fp, ".data\n");

    // 3. Code Segment
    fprintf(fp, ".code\n");

    // 4. Entry point label
    fprintf(fp, "start_program:\n");

    // 5. CRITICAL: Initialize Data Segment
    fprintf(fp, "    MOV AX, @data\n");
    fprintf(fp, "    MOV DS, AX\n");

    // 6. Call compiled main function
    fprintf(fp, "    CALL main\n");

    // 7. Exit to DOS
    fprintf(fp, "    MOV AX, 4C00h\n");
    fprintf(fp, "    INT 21h\n\n");

}

int code_gen(ASTNode *node, FILE *fp) {
    if (!node) return -1;

    int r1, r2, lbl1, lbl2;

    switch(node->kind) {

        case NODE_LITERAL_INT:
            r1 = get_free_reg(fp);
            fprintf(fp, "    MOV %s, %d\n", reg_names[r1], node->int_val);
            return r1;

        case NODE_VAR_ACCESS:
            {
                // We need to look up the symbol to get the offset
                Symbol *s = sym_lookup(node->name);
                if (!s) {
                    fprintf(fp, "; Error: Var %s not found\n", node->name);
                    return get_free_reg(fp);
                }
                // Use Smart Load
                return get_reg_for_var(s->name, s->offset, fp);
            }

        case NODE_ASSIGN:
            // 1. RHS
            r1 = code_gen(node->right, fp);
            // 2. LHS Address
            if(node->left->kind == NODE_VAR_ACCESS) {
                Symbol *s = sym_lookup(node->left->name);
```

```
                if(s) {
                    if(s->offset >= 0)
                        fprintf(fp, "    MOV [BP+%d], %s\n", s->offset,
reg_names[r1]);
                    else
                        fprintf(fp, "    MOV [BP%d], %s\n", s->offset,
reg_names[r1]);

                    // Update descriptor: r1 now contains 's->name'
                    if(regs[r1].content) free(regs[r1].content);
                    regs[r1].content = strdup(s->name);
                }
            }
            free_reg(r1);
            return -1;

        case NODE_EXPR_BINOP:
            r1 = code_gen(node->left, fp);
            r2 = code_gen(node->right, fp);

            // Operation invalidates the content of the destination register
            if(regs[r1].content) { free(regs[r1].content); regs[r1].content = NULL;
}

            if(strcmp(node->op, "+")==0) fprintf(fp, "    ADD %s, %s\n",
reg_names[r1], reg_names[r2]);
            else if(strcmp(node->op, "-")==0) fprintf(fp, "    SUB %s, %s\n",
reg_names[r1], reg_names[r2]);
            else if(strcmp(node->op, "*")==0) fprintf(fp, "    IMUL %s, %s\n",
reg_names[r1], reg_names[r2]);
            else if(strcmp(node->op, ">")==0) {
                 fprintf(fp, "    CMP %s, %s\n", reg_names[r1], reg_names[r2]);
                 fprintf(fp, "    SETG AL\n    CBW\n"); // Result in AX
                 if(r1 != 0) fprintf(fp, "    MOV %s, AX\n", reg_names[r1]);
            }
            free_reg(r2);
            return r1;

        case NODE_FUNC_DEF:
            // Re-bind global scope pointer for symbol lookups inside this function
            // (In a full implementation, we'd manage a stack of scopes)
            if(node->scope) currentScope = node->scope;

            fprintf(fp, "\n%s:\n", node->name);
            fprintf(fp, "    PUSH BP\n");
            fprintf(fp, "    MOV BP, SP\n");

            // BEST PRACTICE: Use the pre-calculated stack size!
            if (node->scope && node->scope->totalStackSize > 0) {
                fprintf(fp, "    SUB SP, %d ; Reserve locals\n", node->scope-
>totalStackSize);
            }
```

```
            code_gen(node->right, fp); // Body

            // Default return in case user didn't write one
            fprintf(fp, "    MOV SP, BP\n");
            fprintf(fp, "    POP BP\n");
            fprintf(fp, "    RET\n");

            // Restore global scope (hack for this simple architecture)
            if(node->scope) currentScope = node->scope->parent;
            return -1;

        case NODE_RETURN:
            if(node->left) {
                r1 = code_gen(node->left, fp);
                if (r1 != 0) fprintf(fp, "    MOV AX, %s\n", reg_names[r1]); //
Retval in AX
                free_reg(r1);
            }
            fprintf(fp, "    MOV SP, BP\n");
            fprintf(fp, "    POP BP\n");
            fprintf(fp, "    RET\n");
            return -1;

        case NODE_FUNC_CALL:
            // Push args (iterate list)
            {
                int bytes = 0;
                ASTNode *arg = node->left ? node->left->left : NULL; // node->left
is NODE_LIST
                while(arg) {
                    r1 = code_gen(arg->left ? arg->left : arg, fp); // Handle list
wrapper
                    fprintf(fp, "    PUSH %s\n", reg_names[r1]);
                    free_reg(r1);
                    bytes += 2;
                    arg = arg->next;
                }
                fprintf(fp, "    CALL %s\n", node->name);
                fprintf(fp, "    ADD SP, %d\n", bytes);
                r1 = get_free_reg(fp);
                fprintf(fp, "    MOV %s, AX\n", reg_names[r1]);
                return r1;
            }

        case NODE_IF:
            lbl1 = new_label(); lbl2 = new_label();
            r1 = code_gen(node->left, fp);
            fprintf(fp, "    CMP %s, 0\n    JE L_else_%d\n", reg_names[r1], lbl1);
            free_reg(r1);
            code_gen(node->right, fp);
            fprintf(fp, "    JMP L_end_%d\n", lbl2);
```

```
            fprintf(fp, "L_else_%d:\n", lbl1);
            if(node->extra) code_gen(node->extra, fp);
            fprintf(fp, "L_end_%d:\n", lbl2);
            return -1;

        case NODE_BLOCK:
            {
                ASTNode *child = node->left;
                while(child) {
                    code_gen(child, fp);
                    child = child->next;
                }
            }
            return -1;

        case NODE_WRITE:
            r1 = code_gen(node->left, fp);
            fprintf(fp, "    PUSH %s\n    CALL _print_int\n    ADD SP, 2\n",
reg_names[r1]);
            free_reg(r1);
            return -1;

        default:
            if(node->left) code_gen(node->left, fp);
            if(node->right) code_gen(node->right, fp);
            if(node->next) code_gen(node->next, fp);
            return -1;
    }
}

void code_gen_finalize(FILE *fp) {
    // Minimal Integer Printer
    fprintf(fp, "\n_print_int:\n");
    fprintf(fp, "    PUSH BP\n    MOV BP, SP\n");
    fprintf(fp, "    MOV AX, [BP+4]\n");
    fprintf(fp, "    ADD AX, '0' ; Hack: only works for single digit 0-9\n");
    fprintf(fp, "    MOV DX, AX\n    MOV AH, 02h\n    INT 21h\n");
    fprintf(fp, "    POP BP\n    RET\n");
}
```

Code: symbol_table.h

```
#ifndef SYMBOL_TABLE_H
#define SYMBOL_TABLE_H

#include <stdio.h>
#include <stdlib.h>
#include "ast.h"

// Symbol Kinds
```

```c
typedef enum {
    KIND_VAR,
    KIND_FUNC,
    KIND_CLASS,
    KIND_PARAM
} SymbolKind;

// Symbol Node
typedef struct Symbol {
    char *name;
    char *type; // "integer", "float", "MyClass", etc.
    SymbolKind kind;
    int offset;
    struct Symbol *next; // Linked list for hash collisions
} Symbol;

// Scope Node
typedef struct Scope {
    int id;
    char *name;
    struct Symbol **table;
    struct Scope *parent;
    struct Scope *next;

    // --- STACK FRAME MANAGEMENT ---
    int paramOffset;     // Counts up (arguments): +4, +6...
    int localOffset;     // Counts down (locals): -2, -4...
    int totalStackSize;  // Final size of locals to reserve (SUB SP, N)
} Scope;

// Core Functions
void sym_init();
void sym_enter_scope(char *name);
void sym_exit_scope();
void sym_insert(char *name, char *type, SymbolKind kind);
Symbol* sym_lookup(char *name);
char* sym_lookup_type(char *name);
void sym_print(char *filename);

// The Merged "Build & Check" Function
void semantic_analysis(ASTNode *root);

void calculate_stack_offsets(ASTNode *funcNode);


#endif
```

Code: symbol_table.c

```c
#include "symbol_table.h"
```

```c
#include <string.h>
#include <stdio.h>

#define HASH_SIZE 211

// Global State
Scope *currentScope = NULL;
Scope *scopeListHead = NULL;
int scopeCount = 0;

// Error Reporting File
FILE *errFile = NULL;
int errorCount = 0;

// --- HASH FUNCTION ---
unsigned int hash(char *s) {
    unsigned int h = 5381;
    int c;
    if (!s) return 0;
    while ((c = *s++)) h = ((h << 5) + h) + c;
    return h % HASH_SIZE;
}

// --- ERROR HELPER ---
void log_error(int lineno, const char *msg, const char *detail) {
    if (!errFile) return;
    fprintf(errFile, "Line %d: Semantic Error: %s %s\n", lineno, msg, detail ?
detail : "");
    printf("Line %d: Semantic Error: %s %s\n", lineno, msg, detail ? detail : "");
    errorCount++;
}

// --- SYMBOL TABLE OPS ---

void sym_init() {
    scopeListHead = NULL;
    currentScope = NULL;
    scopeCount = 0;
    sym_enter_scope("Global");
}

void sym_enter_scope(char *name) {
    Scope *s = (Scope*)malloc(sizeof(Scope));
    s->id = scopeCount++;
    s->name = strdup(name ? name : "unnamed");
    s->table = (struct Symbol**)calloc(HASH_SIZE, sizeof(Symbol*));
    s->parent = currentScope;

    // Maintain global list for printing
    s->next = scopeListHead;
    scopeListHead = s;
```

```
    // --- OFFSET INITIALIZATION (8086 Model) ---
    // BP+0 = Old BP
    // BP+2 = Return Address
    // BP+4 = First Parameter
    s->paramOffset = 4;

    // BP-0 = Old BP (start)
    // BP-2 = First Local
    s->localOffset = -2;
    s->totalStackSize = 0;

    currentScope = s;
}

void sym_exit_scope() {
    if (currentScope) {
        // Calculate total size used by locals (convert negative offset to positive
size)
        // e.g., if localOffset reached -6, we used 4 bytes (starts at -2) + 2
bytes.
        // Math: abs(localOffset + 2) is wrong.
        // Logic: If localOffset is -6, we used -2 and -4. Size is 4.
        currentScope->totalStackSize = -(currentScope->localOffset) - 2;
        if (currentScope->totalStackSize < 0) currentScope->totalStackSize = 0;

        currentScope = currentScope->parent;
    }
}

void sym_insert(char *name, char *type, SymbolKind kind) {
    if (!currentScope || !name) return;
    unsigned int h = hash(name);

    Symbol *sym = (Symbol*)malloc(sizeof(Symbol));
    sym->name = strdup(name);
    sym->type = type ? strdup(type) : strdup("unknown");
    sym->kind = kind;
    sym->next = currentScope->table[h];
    currentScope->table[h] = sym;

    // --- OFFSET CALCULATION ---
    // Assuming 16-bit (2 byte) integers/pointers for simplicity
    int width = 2;

    if (kind == KIND_PARAM) {
        sym->offset = currentScope->paramOffset;
        currentScope->paramOffset += width;
    }
    else if (kind == KIND_VAR) {
        sym->offset = currentScope->localOffset;
        currentScope->localOffset -= width;
    }
```

```c
    else {
        sym->offset = 0; // Funcs/Classes don't have stack offsets
    }

}


// Lookup up the scope chain
Symbol* sym_lookup(char *name) {
    if (!name) return NULL;
    Scope *s = currentScope;
    unsigned int h = hash(name);

    while (s) {
        Symbol *sym = s->table[h];
        while (sym) {
            if (strcmp(sym->name, name) == 0) return sym;
            sym = sym->next;
        }
        s = s->parent;
    }
    return NULL;
}

// Helper to check duplicates in CURRENT scope only
int sym_exists_in_current_scope(char *name) {
    if (!currentScope || !name) return 0;
    unsigned int h = hash(name);
    Symbol *sym = currentScope->table[h];
    while (sym) {
        if (strcmp(sym->name, name) == 0) return 1;
        sym = sym->next;
    }
    return 0;
}

char* sym_lookup_type(char *name) {
    Symbol *s = sym_lookup(name);
    return s ? s->type : NULL;
}

// --- TYPE INFERENCE HELPER ---
char* infer_type(ASTNode *node) {
    if (!node) return "void";

    if (node->kind == NODE_LITERAL_INT) return "integer";
    if (node->kind == NODE_LITERAL_FLOAT) return "float";

    if (node->kind == NODE_VAR_ACCESS) {
        // Handle "self" or basic IDs
        if (strcmp(node->name, "self") == 0) return "class_instance";
        // Handle dot notation (simplified: return unknown or assume last part)
```

```
        if (strchr(node->name, '.')) return "unknown";

        char *t = sym_lookup_type(node->name);
        return t ? t : "unknown";
    }

    if (node->kind == NODE_EXPR_BINOP) {
        // If Logic Op, return integer/boolean
        if (strcmp(node->op, "==") == 0 || strcmp(node->op, "<") == 0 ||
            strcmp(node->op, "or") == 0) return "integer";

        // Otherwise, inherit from left child (Simplified)
        return infer_type(node->left);
    }

    if (node->kind == NODE_FUNC_CALL) {
        // Lookup function return type
        char *t = sym_lookup_type(node->name);
        return t ? t : "unknown";
    }

    return "unknown";
}

// ============================================================
// MERGED TRAVERSAL: BUILD TABLE + CHECK SEMANTICS
// ============================================================

void traverse_and_check(ASTNode *node) {
    if (!node) return;

    int popped = 0;

    // 1. DECLARATION HANDLING
    if (node->kind == NODE_CLASS_DECL) {
        // Check Duplicate Class
        if (sym_exists_in_current_scope(node->name)) {
            log_error(node->lineno, "Duplicate class declaration:", node-
>name);

        } else {
            sym_insert(node->name, "class", KIND_CLASS);
        }

        // Enter Scope
        sym_enter_scope(node->name);
        node->scope = currentScope; // LINK AST TO SCOPE
        popped = 1;
    }
    else if (node->kind == NODE_FUNC_DEF) {
        // Check Duplicate Function
        if (sym_exists_in_current_scope(node->name)) {
```

```
            log_error(node->lineno, "Duplicate function declaration:", node->name);
        } else {
            // Note: In strict Pascal/C, func name goes in parent scope.
            // We insert into parent scope (already there), THEN enter new scope.
            sym_insert(node->name, "function", KIND_FUNC);
        }

        sym_enter_scope(node->name);
        // *** CRITICAL: LINK AST NODE TO THIS NEW SCOPE ***
        node->scope = currentScope;
        popped = 1;
    }
    else if (node->kind == NODE_VAR_DECL || node->kind == NODE_PARAM) {
        // Check Duplicate Variable
        if (sym_exists_in_current_scope(node->name)) {
            log_error(node->lineno, "Duplicate variable/param declaration:", node-
>name);
        } else {
            sym_insert(node->name, node->type_name,
                       node->kind == NODE_PARAM ? KIND_PARAM : KIND_VAR);
        }
    }

    // 2. USAGE CHECKS
    if (node->kind == NODE_VAR_ACCESS) {
        // Skip 'self' and complex dot access for this simplified phase
        if (strcmp(node->name, "self") != 0 && strchr(node->name, '.') == NULL) {
            Symbol *s = sym_lookup(node->name);
            if (!s) {
                log_error(node->lineno, "Undeclared variable:", node->name);
            }
        }
    }

    if (node->kind == NODE_FUNC_CALL) {
         Symbol *s = sym_lookup(node->name);
         if (!s) {
             log_error(node->lineno, "Call to undeclared function:", node->name);
         }
    }

    // 3. TYPE CHECKS (Assignments)
    if (node->kind == NODE_ASSIGN) {
        char *lhsType = infer_type(node->left);
        char *rhsType = infer_type(node->right);

        // If either is unknown, we probably already reported an undeclared error
        if (strcmp(lhsType, "unknown") != 0 && strcmp(rhsType, "unknown") != 0) {
            // Strict match check
            if (strcmp(lhsType, rhsType) != 0) {
                // Allow int-to-float promotion
```

```
                if (!(strcmp(lhsType, "float") == 0 && strcmp(rhsType, "integer") ==
0)) {
                    char msg[100];
                    sprintf(msg, "Type mismatch. Cannot assign '%s' to '%s'",
rhsType, lhsType);
                    log_error(node->lineno, msg, NULL);
                }
            }
        }
    }

    // 4. RECURSION (Depth First)
    traverse_and_check(node->left);
    traverse_and_check(node->right);
    traverse_and_check(node->extra); // Don't forget the 'extra' child (else block)

    // 5. SCOPE EXIT
    if (popped) {
        sym_exit_scope();
    }

    // 6. PROCESS SIBLINGS (Linked Lists)
    traverse_and_check(node->next);
}

// Entry Point
void semantic_analysis(ASTNode *root) {
    errFile = fopen("semantic_errors.txt", "w");
    if (!errFile) {
        perror("Failed to open error file");
        return;
    }

    errorCount = 0;
    sym_init(); // Initialize Global Scope

    // Run the merged pass
    traverse_and_check(root);

    if (errorCount == 0) {
        fprintf(errFile, "Success: No semantic errors found.\n");
    } else {
        fprintf(errFile, "\nTotal Errors: %d\n", errorCount);
    }

    fclose(errFile);
    printf("Semantic Analysis Complete. %d errors found. See semantic_errors.txt\n",
errorCount);
}

void sym_print(char *filename) {
    FILE *fp = fopen(filename, "w");
```

```
    if (!fp) return;

    fprintf(fp,
"==================================================================================
==================\n");
    fprintf(fp, "                                        SYMBOL TABLE
DUMP                                                 \n");
    fprintf(fp,
"==================================================================================
==================\n");

    Scope *s = scopeListHead;
    while (s) {
        // --- HEADER: Added "Stack Size" ---
        fprintf(fp, "\nSCOPE ID: %-3d | NAME: %-20s | PARENT ID: %-3d | STACK SIZE:
%d bytes\n",
                s->id, s->name, s->parent ? s->parent->id : -1, s->totalStackSize);

        fprintf(fp, "+----------------------+------------+-------------------------
-+------------+\n");
        fprintf(fp, "| %-20s | %-10s | %-25s | %-10s |\n", "Symbol Name", "Kind",
"Type", "Offset");
        fprintf(fp, "+----------------------+------------+-------------------------
-+------------+\n");

        for (int i = 0; i < HASH_SIZE; i++) {
            Symbol *sym = s->table[i];
            while (sym) {
                // Determine Kind String
                char *kStr = "UNKNOWN";
                if (sym->kind == KIND_VAR) kStr = "VAR";
                else if (sym->kind == KIND_FUNC) kStr = "FUNC";
                else if (sym->kind == KIND_CLASS) kStr = "CLASS";
                else if (sym->kind == KIND_PARAM) kStr = "PARAM";

                // --- OFFSET FORMATTING ---
                // We format the offset string (e.g., "BP-2", "BP+4", or "-")
                char offsetStr[16];
                if (sym->kind == KIND_VAR) {
                    snprintf(offsetStr, 16, "%d (BP%d)", sym->offset, sym->offset);
                } else if (sym->kind == KIND_PARAM) {
                    snprintf(offsetStr, 16, "%d (BP+%d)", sym->offset, sym->offset);
                } else {
                    snprintf(offsetStr, 16, "-");
                }

                fprintf(fp, "| %-20s | %-10s | %-25s | %-10s |\n",
                        sym->name, kStr, sym->type, offsetStr);

                sym = sym->next;
            }
        }
```

```
        fprintf(fp, "+----------------------+------------+-------------------------
-+------------+\n");
        s = s->next;
    }
    fclose(fp);
    printf("Symbol table dumped to %s\n", filename);
}
```

Code: ast.h

```
#ifndef AST_H
#define AST_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Forward Declaration to break circular dependency
struct Scope;

typedef enum {
    NODE_PROGRAM, NODE_CLASS_DECL, NODE_IMPL_DEF, NODE_FUNC_DEF, NODE_FUNC_HEAD,
    NODE_VAR_DECL, NODE_MEMBER_DECL, NODE_BLOCK, NODE_ASSIGN, NODE_IF,
    NODE_WHILE, NODE_READ, NODE_WRITE, NODE_RETURN, NODE_FUNC_CALL,
    NODE_EXPR_BINOP, NODE_EXPR_UNOP, NODE_VAR_ACCESS, NODE_LITERAL_INT,
    NODE_LITERAL_FLOAT, NODE_TYPE, NODE_PARAM, NODE_LIST
} NodeType;

typedef struct ASTNode {
    NodeType kind;

    // Metadata
    char *name;
    char *type_name;
    char *op;
    int int_val;
    float float_val;
    char *visibility;

    // Structure
    struct ASTNode *left;
    struct ASTNode *right;
    struct ASTNode *extra;
    struct ASTNode *next;

    int lineno;

    // --- NEW: LINK TO SYMBOL TABLE SCOPE ---
    // This allows the Code Generator to know stack size immediately
    struct Scope *scope;
```

```
} ASTNode;

// ... (Rest of your constructors remain the same) ...
ASTNode* createNode(NodeType kind);
ASTNode* createBinary(char *op, ASTNode *left, ASTNode *right);
ASTNode* createUnary(char *op, ASTNode *child);
ASTNode* createID(char *name);
ASTNode* createLiteralInt(int val);
ASTNode* createLiteralFloat(float val);
void ast_print(ASTNode *root, char *filename);

#endif
```

Code: ast.c

```
#include "ast.h"

extern int yylineno;

ASTNode* createNode(NodeType kind) {
    ASTNode *node = (ASTNode*)malloc(sizeof(ASTNode));
    node->kind = kind;
    node->name = NULL;
    node->type_name = NULL;
    node->op = NULL;
    node->visibility = NULL;
    node->left = NULL;
    node->right = NULL;
    node->extra = NULL;
    node->next = NULL;

    // --- NEW ---
    node->scope = NULL;

    node->lineno = yylineno;
    return node;
}

ASTNode* createBinary(char *op, ASTNode *left, ASTNode *right) {
    ASTNode *node = createNode(NODE_EXPR_BINOP);
    node->op = strdup(op);
    node->left = left;
    node->right = right;
    return node;
}

ASTNode* createUnary(char *op, ASTNode *child) {
    ASTNode *node = createNode(NODE_EXPR_UNOP);
    node->op = strdup(op);
    node->left = child;
    return node;
}
```

```c
ASTNode* createID(char *name) {
    ASTNode *node = createNode(NODE_VAR_ACCESS);
    node->name = strdup(name);
    return node;
}

ASTNode* createLiteralInt(int val) {
    ASTNode *node = createNode(NODE_LITERAL_INT);
    node->int_val = val;
    return node;
}

ASTNode* createLiteralFloat(float val) {
    ASTNode *node = createNode(NODE_LITERAL_FLOAT);
    node->float_val = val;
    return node;
}

// ==========================================================
// TREE VISUALIZATION LOGIC
// ==========================================================

void print_branch(FILE *fp, ASTNode *node, char *prefix, int isLast) {
    if (!node) return;

    // 1. Print the current node's prefix and connection
    fprintf(fp, "%s", prefix);
    fprintf(fp, "%s", isLast ? "\\-- " : "|-- ");

    // 2. Print the Node's Content
    switch (node->kind) {
        case NODE_PROGRAM:      fprintf(fp, "PROGRAM\n"); break;
        case NODE_CLASS_DECL:   fprintf(fp, "CLASS [Name: %s]\n", node->name);
break;
        case NODE_IMPL_DEF:     fprintf(fp, "IMPLEMENT [Name: %s]\n", node->name);
break;
        case NODE_FUNC_DEF:     fprintf(fp, "FUNCTION [Name: %s]\n", node->name);
break;
        case NODE_FUNC_HEAD:    fprintf(fp, "SIGNATURE [Name: %s]\n", node->name);
break;
        case NODE_BLOCK:        fprintf(fp, "BLOCK\n"); break;
        case NODE_VAR_DECL:     fprintf(fp, "VAR [Name: %s, Type: %s]\n", node-
>name, node->type_name); break;
        case NODE_PARAM:        fprintf(fp, "PARAM [Name: %s, Type: %s]\n", node-
>name, node->type_name); break;
        case NODE_ASSIGN:       fprintf(fp, "ASSIGN (:=)\n"); break;
        case NODE_IF:           fprintf(fp, "IF\n"); break;
        case NODE_WHILE:        fprintf(fp, "WHILE\n"); break;
        case NODE_RETURN:       fprintf(fp, "RETURN\n"); break;
        case NODE_READ:         fprintf(fp, "READ\n"); break;
        case NODE_WRITE:        fprintf(fp, "WRITE\n"); break;
```

```
        case NODE_EXPR_BINOP:   fprintf(fp, "OP (%s)\n", node->op); break;
        case NODE_EXPR_UNOP:    fprintf(fp, "UNARY OP (%s)\n", node->op); break;
        case NODE_FUNC_CALL:    fprintf(fp, "CALL [Func: %s]\n", node->name); break;
        case NODE_VAR_ACCESS:   fprintf(fp, "ID [%s]\n", node->name); break;
        case NODE_LITERAL_INT:  fprintf(fp, "INT [%d]\n", node->int_val); break;
        case NODE_LITERAL_FLOAT:fprintf(fp, "FLOAT [%f]\n", node->float_val); break;
        case NODE_TYPE:         fprintf(fp, "TYPE [%s]\n", node->name); break;
        case NODE_LIST:         fprintf(fp, "LIST\n"); break;
        default:                fprintf(fp, "UNKNOWN NODE (%d)\n", node->kind);
    }

    // 3. Prepare prefix for children
    char newPrefix[256];
    snprintf(newPrefix, sizeof(newPrefix), "%s%s", prefix, isLast ? "    " :
"|   ");

    // 4. Collect non-null children pointers to process
    //    (We handle 'next' separately in the loop, so here we only look at
structural children)
    ASTNode *children[3] = {node->left, node->right, node->extra};
    char *labels[3] = {"Left", "Right", "Extra"};

    // Count actual structural children to know which is last
    int childCount = 0;
    for(int i=0; i<3; i++) if(children[i]) childCount++;

    int currentChild = 0;
    for(int i=0; i<3; i++) {
        if(children[i]) {
            currentChild++;
            int isLastChild = (currentChild == childCount);

            // If the child is a list (has 'next'), we traverse the list here
            // effectively treating the whole list as children of the current node
            ASTNode *temp = children[i];

            // Print label (optional, helps disambiguate IF/ELSE etc)
            // fprintf(fp, "%s[%s]\n", newPrefix, labels[i]);

            while(temp) {
                // If it's the last node in the list AND the last child group
                int isListLast = (temp->next == NULL) && isLastChild;
                print_branch(fp, temp, newPrefix, isListLast);
                temp = temp->next;
            }
        }
    }
}

void ast_print(ASTNode *root, char *filename) {
    FILE *fp = fopen(filename, "w");
    if (!fp) {
```

```
        perror("Cannot open output file");
        return;
    }
    fprintf(fp, "ABSTRACT SYNTAX TREE\n");
    fprintf(fp, "====================\n");

    // Start with empty prefix
    // Handle the root list (e.g., list of classes/funcs)
    ASTNode *temp = root;
    while(temp) {
        print_branch(fp, temp, "", temp->next == NULL);
        temp = temp->next;
    }

    fclose(fp);
    printf("AST written to %s\n", filename);
}
```

Code: lexer.l

```
%{
#include "ast.h"
#include "parser.tab.h"

extern int yylineno;

// Standard library includes for string operations (e.g., strdup, atoi, atof)
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

// NOTE: You would typically define the yylval union here for semantic value
transfer.
// For simplicity, we are returning the token code directly.
%}

/* * Regular Definitions Section
 * These are substituted into the Rules section below.
 */
letter      [a-zA-Z]
digit       [0-9]
nonzero     [1-9]
alphanum    ({letter}|{digit}|_)
integer     (0|{nonzero}{digit}*)
fraction    (\.({digit})*{nonzero}|\.0)
float       {integer}{fraction}([eE][+-]?{integer})?
comment     (\/\/.*|\/\*([^*]|\*+[^*/])*\*+\/)
```

```
%%

{comment}         {};

[ \t\r]+            ;
\n                  { yylineno++; }



"=>"                { return ARROW_OP; }
":="                { return ASSIGN_OP; }
"=="                { return EQ_OP; }
"<>"                { return NE_OP; }
"<="                { return LE_OP; }
">="                { return GE_OP; }
"or"                { return OR; }
"and"               { return AND; }
"not"               { return NOT; }



"else"              { return ELSE; }
"float"             { return FLOAT; }
"func"              { return FUNC; }
"if"                { return IF; }
"implement"         { return IMPLEMENT; }
"class"             { return CLASS; }
"attribute"         { return ATTRIBUTE; }
"integer"           { return INTEGER; }
"isa"               { return ISA; }
"private"           { return PRIVATE; }
"public"            { return PUBLIC; }
"read"              { return READ; }
"return"            { return RETURN; }
"self"              { return SELF; }
"construct"         { return CONSTRUCT; }
"then"              { return THEN; }
"local"             { return LOCAL; }
"void"              { return VOID; }
"while"             { return WHILE; }
"write"             { return WRITE; }



{integer}   { yylval.ival = atoi(yytext); return LITERAL_INTEGER; }
{float}     { yylval.fval = atof(yytext); return LITERAL_FLOAT; }

{letter}({alphanum})* {
    yylval.sval = strdup(yytext);
    return ID;
}
```

```
"("                 { return '('; }
")"                 { return ')'; }
"{"                 { return '{'; }
"}"                 { return '}'; }
"["                 { return '['; }
"]"                 { return ']'; }
";"                 { return ';'; }
","                 { return ','; }
"."                 { return '.'; }
":"                 { return ':'; }
"+"                 { return '+'; }
"-"                 { return '-'; }
"*"                 { return '*'; }
"/"                 { return '/'; }
"<"                 { return '<'; }
">"                 { return '>'; }


.                   {fprintf(stderr, "Lexical Error: Unexpected character '%s' at
line %d\n", yytext, yylineno);}
%%

/* * Auxiliary Functions Section
 * This is where you would define the main() function to drive the scanner
 * and any required error handling functions.
 */

int yywrap() {
    return 1;
}
```

Code: parser.y

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "ast.h"
#include "symbol_table.h"
#include "code_gen.h"

// Forward declarations
int yylex(void);
void yyerror(const char *s);
extern int yylineno;
```

```
// Global root for the AST
ASTNode *rootNode = NULL;
%}

/* Union for passing data between Lexer and Parser */
%union {
    int ival;
    float fval;
    char *sval;
    struct ASTNode *node;
}

/* Tokens */
%token CLASS IMPLEMENT FUNC ATTRIBUTE
%token PUBLIC PRIVATE LOCAL
%token IF THEN ELSE WHILE
%token READ WRITE RETURN
%token ISA
%token INTEGER FLOAT VOID
%token SELF
%token CONSTRUCT

%token <sval> ID
%token <ival> LITERAL_INTEGER
%token <fval> LITERAL_FLOAT

%token OR AND NOT
%token ARROW_OP ASSIGN_OP
%token EQ_OP NE_OP LE_OP GE_OP

/* Non-terminal types - CRITICAL: Everything returns a node */
%type <node> prog classOrImplOrFunc_ZM classOrImplOrFunc
%type <node> classDecl isa_Id_ZO commaID_ZM visibility_memberDecl_ZM
%type <node> implDef funcDef_ZM funcDef funcHead funcBody
%type <node> visibility memberDecl funcDecl attributeDecl
%type <node> localVarDecl varDecl arraySize_ZM arraySize
%type <node> varDeclOrStmt_ZM varDeclOrStmt statement matchedStmt unmatchedStmt
statement_ZM
%type <node> expr relExpr arithExpr term factor
%type <node> variable primaryVar idOrSelf suffix_ZM suffix indice_ZM indice
%type <node> functionCall fParams fParamsTail_ZM fParamsTail aParams aParamsTail_ZM
aParamsTail
%type <node> type returnType assignOp relOp addOp multOp

/* Precedence */
%left OR
%left AND
%left EQ_OP NE_OP '<' '>' LE_OP GE_OP
%left '+' '-'
%left '*' '/'
%right NOT
%right UMINUS
```

```
%start prog

%%

/* ================= PROGRAM ================= */

prog
    : classOrImplOrFunc_ZM
      { rootNode = $1; }
    ;

classOrImplOrFunc_ZM
    : classOrImplOrFunc classOrImplOrFunc_ZM
      {
        $$ = $1;
        if ($$ != NULL) $$->next = $2;
        else $$ = $2; // Logic safety
      }
    | /* ε */
      { $$ = NULL; }
    ;

classOrImplOrFunc
    : classDecl { $$ = $1; }
    | implDef   { $$ = $1; }
    | funcDef   { $$ = $1; }
    ;

/* ================= CLASS ================= */

classDecl
    : CLASS ID isa_Id_ZO '{' visibility_memberDecl_ZM '}' ';'
      {
        $$ = createNode(NODE_CLASS_DECL);
        $$->name = $2;
        $$->left = $3;  // Inheritance list
        $$->right = $5; // Members list
      }
    ;

isa_Id_ZO
    : ISA ID commaID_ZM
      {
         $$ = createNode(NODE_LIST); // Placeholder for inheritance list
         $$->name = $2;
         $$->next = $3;
      }
    | /* ε */ { $$ = NULL; }
    ;

commaID_ZM
```

```
    : ',' ID commaID_ZM
      {
         ASTNode *n = createNode(NODE_LIST);
         n->name = $2;
         n->next = $3;
         $$ = n;
      }
    | /* ε */ { $$ = NULL; }
    ;

visibility_memberDecl_ZM
    : visibility memberDecl visibility_memberDecl_ZM
      {
        // memberDecl ($2) is the content. We attach visibility ($1) to it.
        // But $2 might already be constructed. Let's modify $2's visibility field.
        $$ = $2;
        if ($$) {
             $$->visibility = $1->name; // Transfer "public"/"private" string
             $$->next = $3;
        }
      }
    | /* ε */ { $$ = NULL; }
    ;

/* ================= IMPLEMENT ================= */

implDef
    : IMPLEMENT ID '{' funcDef_ZM '}'
      {
        $$ = createNode(NODE_IMPL_DEF);
        $$->name = $2;
        $$->right = $4; // Functions
      }
    ;

funcDef_ZM
    : funcDef funcDef_ZM
      {
        $$ = $1;
        if($$) $$->next = $2;
      }
    | /* ε */ { $$ = NULL; }
    ;

/* ================= FUNCTIONS ================= */

funcDef
    : funcHead funcBody
      {
        // funcHead returns a node with signature info.
        // We wrap it in a FUNC_DEF node.
        $$ = createNode(NODE_FUNC_DEF);
```

```
        $$->name = $1->name;       // Extract name from head
        $$->left = $1;             // Keep head details (params/type) as left child
        $$->right = $2;            // Body as right child
      }
    ;

funcHead
    : FUNC ID '(' fParams ')' ARROW_OP returnType
      {
        $$ = createNode(NODE_FUNC_HEAD);
        $$->name = $2;
        $$->left = $4; // Params
        $$->right = $7; // Return type
      }
    | CONSTRUCT '(' fParams ')'
      {
        $$ = createNode(NODE_FUNC_HEAD);
        $$->name = strdup("construct");
        $$->left = $3;
        $$->right = NULL; // No return type
      }
    ;

funcBody
    : '{' varDeclOrStmt_ZM '}' { $$ = $2; } // Body is just a list of stmts
    ;

visibility
    : PUBLIC  { $$ = createNode(NODE_TYPE); $$->name = strdup("public"); }
    | PRIVATE { $$ = createNode(NODE_TYPE); $$->name = strdup("private"); }
    ;

memberDecl
    : funcDecl { $$ = $1; }
    | attributeDecl { $$ = $1; }
    ;

funcDecl
    : funcHead ';'
      {
         $$ = $1; // Just return the head info
      }
    ;

/* ================= VARIABLES ================= */

attributeDecl
    : ATTRIBUTE varDecl
      {
         $$ = $2;
         // Mark as attribute if needed, or rely on context in class
      }
```

```
    ;

localVarDecl
    : LOCAL varDecl { $$ = $2; }
    ;

varDecl
    : ID ':' type arraySize_ZM ';'
      {
        $$ = createNode(NODE_VAR_DECL);
        $$->name = $1;
        $$->type_name = $3->name; // Extract type string
        $$->left = $4; // Array dimensions
      }
    ;

arraySize_ZM
    : arraySize arraySize_ZM
      { $$ = $1; $$->next = $2; }
    | /* ε */ { $$ = NULL; }
    ;

arraySize
    : '[' LITERAL_INTEGER ']'
      {
        $$ = createNode(NODE_LITERAL_INT);
        $$->int_val = $2;
      }
    | '[' ']'
      {
        $$ = createNode(NODE_LITERAL_INT);
        $$->int_val = -1; // -1 represents empty []
      }
    ;

/* ================= STATEMENTS ================= */

varDeclOrStmt_ZM
    : varDeclOrStmt varDeclOrStmt_ZM
      { $$ = $1; if($$) $$->next = $2; }
    | /* ε */ { $$ = NULL; }
    ;

varDeclOrStmt
    : localVarDecl { $$ = $1; }
    | statement    { $$ = $1; }
    ;

statement
    : matchedStmt { $$ = $1; }
    | unmatchedStmt { $$ = $1; }
    ;
```

```
matchedStmt
    : variable assignOp expr ';'
      {
        $$ = createNode(NODE_ASSIGN);
        $$->left = $1;
        $$->right = $3;
      }
    | functionCall ';' { $$ = $1; }
    | IF '(' relExpr ')' THEN matchedStmt ELSE matchedStmt ';'
      {
        $$ = createNode(NODE_IF);
        $$->left = $3;  // Condition
        $$->right = $6; // Then block
        $$->extra = $8; // Else block
      }
    | WHILE '(' relExpr ')' matchedStmt ';'
      {
        $$ = createNode(NODE_WHILE);
        $$->left = $3;
        $$->right = $5;
      }
    | READ '(' variable ')' ';'
      {
        $$ = createNode(NODE_READ);
        $$->left = $3;
      }
    | WRITE '(' expr ')' ';'
      {
        $$ = createNode(NODE_WRITE);
        $$->left = $3;
      }
    | RETURN '(' expr ')' ';'
      {
        $$ = createNode(NODE_RETURN);
        $$->left = $3;
      }
    | '{' statement_ZM '}'
      {
        $$ = createNode(NODE_BLOCK);
        $$->left = $2;
      }
    ;

unmatchedStmt
    : IF '(' relExpr ')' THEN statement
      {
        $$ = createNode(NODE_IF);
        $$->left = $3;
        $$->right = $6;
        $$->extra = NULL; // No else
      }
```

```
    | IF '(' relExpr ')' THEN matchedStmt ELSE unmatchedStmt ';'
      {
        $$ = createNode(NODE_IF);
        $$->left = $3;
        $$->right = $6;
        $$->extra = $8;
      }
    | WHILE '(' relExpr ')' unmatchedStmt ';'
      {
        $$ = createNode(NODE_WHILE);
        $$->left = $3;
        $$->right = $5;
      }
    ;

statement_ZM
    : statement statement_ZM
      { $$ = $1; if($$) $$->next = $2; }
    | /* ε */ { $$ = NULL; }
    ;

/* ================= EXPRESSIONS ================= */

expr
    : relExpr { $$ = $1; }
    ;

relExpr
    : arithExpr relOp arithExpr
      { $$ = createBinary($2->op, $1, $3); }
    | arithExpr { $$ = $1; }
    ;

arithExpr
    : arithExpr addOp term
      { $$ = createBinary($2->op, $1, $3); }
    | term { $$ = $1; }
    ;

term
    : term multOp factor
      { $$ = createBinary($2->op, $1, $3); }
    | factor { $$ = $1; }
    ;

factor
    : variable { $$ = $1; }
    | functionCall { $$ = $1; }
    | LITERAL_INTEGER { $$ = createLiteralInt($1); }
    | LITERAL_FLOAT   { $$ = createLiteralFloat($1); }
    | '(' arithExpr ')' { $$ = $2; }
    | NOT factor { $$ = createUnary("not", $2); }
```

```
    | '-' factor %prec UMINUS { $$ = createUnary("-", $2); }
    | '+' factor %prec UMINUS { $$ = createUnary("+", $2); }
    ;

/* ================= VARIABLE / ID NEST ================= */
/* This is complex: variable -> primaryVar -> suffix_ZM */
/* We will flatten this to a node tree */

variable
    : primaryVar suffix_ZM
      {
        if ($2 == NULL) {
            $$ = $1; // Just the ID
        } else {
            // Suffixes exist (array indexing or dot notation)
            // We attach suffix list to the primary var
            $$ = $1;
            $$->right = $2; // Attach suffixes to right
        }
      }
    ;

primaryVar
    : idOrSelf { $$ = $1; }
    ;

idOrSelf
    : ID   { $$ = createID($1); }
    | SELF { $$ = createID("self"); }
    ;

suffix_ZM
    : suffix suffix_ZM
      { $$ = $1; $$->next = $2; }
    | indice_ZM { $$ = $1; } /* Indices are also suffixes effectively */
    ;

suffix
    : indice_ZM '.' idOrSelf
      {
        // Complex: array access then dot?
        // We'll create a DOT node.
        ASTNode *dot = createBinary(".", NULL, $3);
        if ($1) {
             // If indices exist, they apply to the *left* of the dot (the prev
object)
             // But here we are returning a suffix chain.
             // Strategy: Return a chain node.
             dot->left = $1; // Store indices here
        }
        $$ = dot;
      }
```

```
    | '(' aParams ')' '.' idOrSelf
      {
        // Method call then dot access
        ASTNode *call = createNode(NODE_FUNC_CALL);
        call->left = $2; // params

        ASTNode *dot = createBinary(".", call, $5);
        $$ = dot;
      }
    ;

indice_ZM
    : indice indice_ZM { $$ = $1; $$->next = $2; }
    | /* ε */ { $$ = NULL; }
    ;

indice
    : '[' arithExpr ']'
      {
        // Represent array index as a specialized node
        ASTNode *idx = createNode(NODE_VAR_ACCESS);
        idx->op = strdup("[]");
        idx->left = $2; // The expression inside [ ]
        $$ = idx;
      }
    ;

/* ================= FUNCTION CALL ================= */

functionCall
    : primaryVar '(' aParams ')'
      {
        $$ = createNode(NODE_FUNC_CALL);
        $$->name = $1->name; // Function name
        $$->left = $3;       // Params
      }
    ;

/* ================= PARAMETERS ================= */

fParams
    : ID ':' type arraySize_ZM fParamsTail_ZM
      {
         $$ = createNode(NODE_PARAM);
         $$->name = $1;
         $$->type_name = $3->name;
         $$->left = $4; // Array size info
         $$->next = $5; // Next param
      }
    | /* ε */ { $$ = NULL; }
    ;
```

```
fParamsTail_ZM
    : fParamsTail fParamsTail_ZM { $$ = $1; $$->next = $2; }
    | /* ε */ { $$ = NULL; }
    ;

fParamsTail
    : ',' ID ':' type arraySize_ZM
      {
         $$ = createNode(NODE_PARAM);
         $$->name = $2;
         $$->type_name = $4->name;
         $$->left = $5;
      }
    ;

aParams
    : expr aParamsTail_ZM
      {
        $$ = createNode(NODE_LIST); // Wrapper for arg list
        $$->left = $1;
        $$->next = $2;
      }
    | /* ε */ { $$ = NULL; }
    ;

aParamsTail_ZM
    : aParamsTail aParamsTail_ZM { $$ = $1; $$->next = $2; }
    | /* ε */ { $$ = NULL; }
    ;

aParamsTail
    : ',' expr
      {
        $$ = createNode(NODE_LIST);
        $$->left = $2;
      }
    ;

/* ================ TYPES & OPS ================ */

type
    : INTEGER { $$ = createNode(NODE_TYPE); $$->name = strdup("integer"); }
    | FLOAT   { $$ = createNode(NODE_TYPE); $$->name = strdup("float"); }
    | ID      { $$ = createNode(NODE_TYPE); $$->name = $1; }
    ;

returnType
    : type { $$ = $1; }
    | VOID { $$ = createNode(NODE_TYPE); $$->name = strdup("void"); }
    ;

assignOp
```

```
    : ASSIGN_OP { $$ = NULL; }
    ;

relOp
    : EQ_OP { $$ = createNode(NODE_TYPE); $$->op = strdup("=="); }
    | '<'   { $$ = createNode(NODE_TYPE); $$->op = strdup("<"); }
    | '>'   { $$ = createNode(NODE_TYPE); $$->op = strdup(">"); }
    | NE_OP { $$ = createNode(NODE_TYPE); $$->op = strdup("<>"); }
    | LE_OP { $$ = createNode(NODE_TYPE); $$->op = strdup("<="); }
    | GE_OP { $$ = createNode(NODE_TYPE); $$->op = strdup(">="); }
    ;

addOp
    : '+' { $$ = createNode(NODE_TYPE); $$->op = strdup("+"); }
    | '-' { $$ = createNode(NODE_TYPE); $$->op = strdup("-"); }
    | OR  { $$ = createNode(NODE_TYPE); $$->op = strdup("or"); }
    ;

multOp
    : '*' { $$ = createNode(NODE_TYPE); $$->op = strdup("*"); }
    | '/' { $$ = createNode(NODE_TYPE); $$->op = strdup("/"); }
    | AND { $$ = createNode(NODE_TYPE); $$->op = strdup("and"); }
    ;

%%

void yyerror(const char *s) {
    fprintf(stderr, "Syntax Error at line %d: %s\n", yylineno, s);
}

int main(void) {
    if (yyparse() == 0) {
        printf("Parsing Successful.\n");

        // ---------------------------------------------------------
        // PHASE 1: FRONTEND VISUALIZATION (From Old)
        // ---------------------------------------------------------
        // Helpful to ensure the Tree is built correctly before processing
        ast_print(rootNode, "ast.txt");

        // ---------------------------------------------------------
        // PHASE 2: SEMANTICS & MEMORY LAYOUT (Shared)
        // ---------------------------------------------------------
        // Builds symbol table, checks errors, calculates Stack Offsets
        semantic_analysis(rootNode);

        // ---------------------------------------------------------
        // PHASE 3: SYMBOL TABLE DUMP (From Old)
        // ---------------------------------------------------------
        // CRITICAL: Check this file to see if 'Offset' and 'Size'
        // columns are correct before looking at the Assembly.
        sym_print("symbol_table.txt");
```

```
        // ---------------------------------------------------------
        // PHASE 4: CODE GENERATION (From New)
        // ---------------------------------------------------------
        // Only generate code if we passed semantic checks.
        // (Assuming semantic_analysis sets a global error flag,
        //  but for now we run it regardless).

        FILE *asmFile = fopen("output.asm", "w");
        if (!asmFile) {
            perror("Failed to open output.asm");
            return 1;
        }

        code_gen_init(asmFile);

        // Traverse top-level nodes (Classes, Funcs, etc.)
        ASTNode *curr = rootNode;
        while(curr) {
            code_gen(curr, asmFile);
            curr = curr->next;
        }

        code_gen_finalize(asmFile);
        fclose(asmFile);

        printf("Compilation Complete. \n");
        printf(" - AST: ast.txt\n");
        printf(" - Symbols: symbol_table.txt\n");
        printf(" - Assembly: output.asm\n");

    } else {
        printf("Parsing Failed.\n");
    }
    return 0;
}
```

References

Gemini AI is used during the system planning, Code refining, Code Comment optimization, and proofreading.

A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.